The HLS scheduling contract



What hardware architects and verification engineers
should know about the Catapult HLS user-view scheduling rules

Stuart Swan
Platform Architect, Siemens EDA

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

A small contract that lets verification stay at the source

HLS may add latency, pipeline loops and reorder memory accesses. The scheduling contract states exactly what it may not change — so a testbench written for the SystemC model stays meaningful for the RTL.



The goal

“No surprises”: verify and debug mostly on the pre-HLS model, with one testbench for both models.



The rules

Three classes of IO — message passing, signal IO and synchronization — each with one uniform rule.



The freedom

Pipelining, added states, late sampling of stable inputs, permitted memory reordering, finite buffering.



The guarantees

Safety: covered RTL behavior is matched by the reference model. Completion: required outputs arrive.



Your part

Latency-insensitive interfaces, well-anchored signal IO, explicit memory, reset and channel structure.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Agenda

The why

1 Why a scheduling contract

Motivation, analogies, latency insensitivity

2 The vocabulary

IO classes, issue and commit

The rules

3 The basic conceptual model

Four rules for every process

4 Optimizations within the contract

Pipelining, direct inputs, memories, capacity

Modeling practice

5 Writing models that stay in scope

RULE 1, RULE 2, mismatch sources, latency-sensitive islands, guidelines

The guarantees

6 Formal guarantees, simplified

Safety, completion and their limits

7 Putting it to work

Design constraints and evidence

8 Conclusion

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

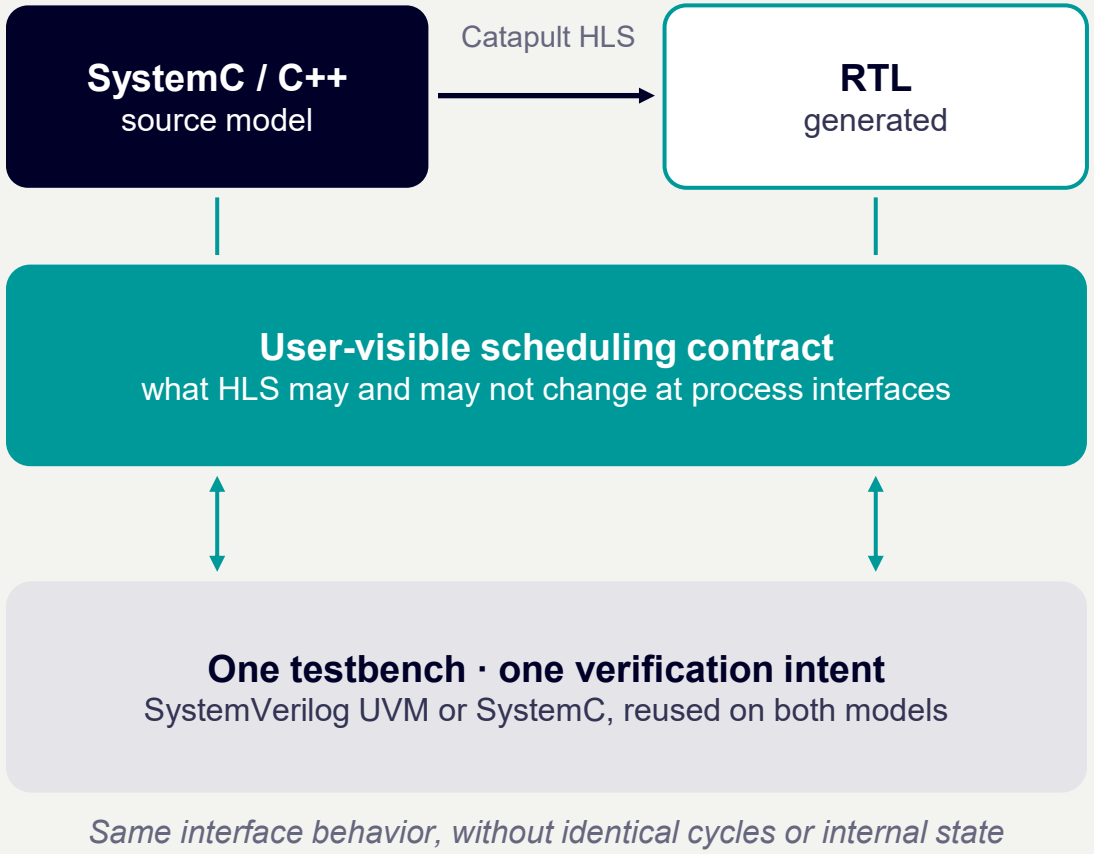
HLS pays off only if verification stays at the source

The problem

- RTL verification is among the most costly and time-consuming parts of a hardware flow.
- Generated RTL can differ substantially from the source model in structure and in timing.
- If teams must return to RTL as the primary level for verification and debug, HLS loses much of its advantage.

The aim

Designers and DV engineers reason primarily at the source level, while keeping a precise way to qualify the resulting implementation.



Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

An analogy from RTL synthesis

In Verilog

```
Out1 <= #some_delay new_val;
```

- Simulation honors the delay. Longer than a clock period, it changes cycle-level behavior.
- RTL synthesis ignores every delay — and may warn: “simulation and synthesis results are likely to mismatch.”
- Nobody expects synthesis to match the simulation exactly. Everyone aligns on the conceptual model.

	RTL synthesis	HLS with this contract
Conceptual model	Clocked RTL semantics	The user-view scheduling rules
Simulation model	Verilog simulation, including #delays	SystemC / C++ simulation of the source
Synthesis	Ignores delays; flags suspicious ones	Schedules within the rules; flags RULE 1 / RULE 2 violations

Why it works: the conceptual model is simple enough for users and tool vendors to align around, so small gaps between simulation and synthesis are easy to manage.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Latency insensitivity is the core design principle

By default, HLS may insert clock cycles anywhere within a process — when pipelining a loop or sharing a resource.

Make the system latency-insensitive

Design and testbench operate correctly despite variable communication delays. AXI4, APB and the AMBA 5 CHI credit-based NoC protocol are familiar examples.

Isolate what is latency-sensitive

Confine fixed-timing protocols to small, self-contained blocks such as transactors, and keep the rest of the design and testbench latency-insensitive.

Check end-to-end latency on RTL

Functional compatibility does not imply equal latency or throughput. Verify latency requirements on the post-HLS model — typically an easy check.

Bonus: with a qualified source realization and timing model, the same latency-insensitive SystemC model can also predict capacity and throughput effects before RTL exists.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

A factory analogy for the rules

People at workbenches	»	**Processes**
Their written instructions	»	**C++ code in a process**
Objects handed between people	»	**Messages (blocking or non-blocking)**
A wall clock everyone can see	»	**The hardware clock**
Flags, seen at the next tick	»	**Signals (delayed update)**
Subordinates doing subtasks	»	**Pipeline stages**
Meeting at a barrier	»	**Explicit synchronization**
Raising a reset flag	»	**Hardware reset**

Now assume

- Task and transfer times vary
- Everyone works as quickly as they can
- Several products — chairs in different colors and styles — are built at once

To produce furniture reliably, everyone must follow rules like those in this contract.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Overlap is harmless; reversing source order can deadlock

```
// Person 1
seats.Push(s);
backs.Push(b);
```

```
// Person 2
s = seats.Pop();
b = backs.Pop();
```

Blocking calls, zero storage between people: a Push completes only when the matching Pop does.

Each transfer takes at least one minute.

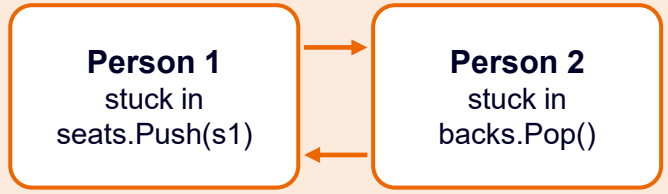
As written: 5 minutes

#	Person 1	Person 2
1	seats.Push(s1)	
2	backs.Push(b1)	s1 = seats.Pop()
3	seats.Push(s2)	b1 = backs.Pop()
4	backs.Push(b2)	s2 = seats.Pop()
5		b2 = backs.Pop()

Both overlap IO: 3 minutes

#	Person 1	Person 2
1	seats.Push(s1) backs.Push(b1)	
2	seats.Push(s2) backs.Push(b2)	s1 = seats.Pop() b1 = backs.Pop()
3		s2 = seats.Pop() b2 = backs.Pop()

Person 2 reverses: deadlock



Person 1 waits for a seats.Pop that never comes; Person 2 waits for a backs.Push that never starts.

Rule of thumb: HLS may overlap message operations, which changes only timing — but it may not reverse their source order.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Key terms

Process

An always block in Verilog; an SC_THREAD or SC_METHOD instance in SystemC. HLS synthesizes each process independently.

Latency-insensitive

Operates correctly despite variable communication delays between components — e.g., AXI4, APB, CHI.

Message-passing interface

Reliable delivery between processes, as in Kahn process networks. Point-to-point: one producer, one consumer.

Synchronization interface

Synchronizes a process with another process and/or the clock, like a barrier: wait(), SyncChannel, ac_sync.

Signal IO

The basic hardware communication mechanism. Modeled with delayed-update semantics: it always costs some delay.

Blocking / non-blocking

Blocking: a persistent request held, with stable payload, until it commits. Non-blocking: returns at once with a status.

One-way / two-way handshake

One signal between sender and receiver (e.g., dat/vld), or one in each direction (vld and rdy).

Shall / may

Shall: required of a compliant tool or flow. May: allowed, but not required.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Three classes of IO operation

Message passing

ac_channel methods; Push, Pop, PushNB, PopNB (e.g., Connections In<> / Out<>)

Moves data between processes. May shift and overlap in time.

Synchronization

wait(), ac_wait, SyncChannel (sync_in / sync_out), ac_sync

Defines the boundaries the other operations respect. Always stays in source order.

Signal IO

sc_signal / sc_in / sc_out reads and writes; C++ direct inputs

Anchored to the surrounding synchronization calls.

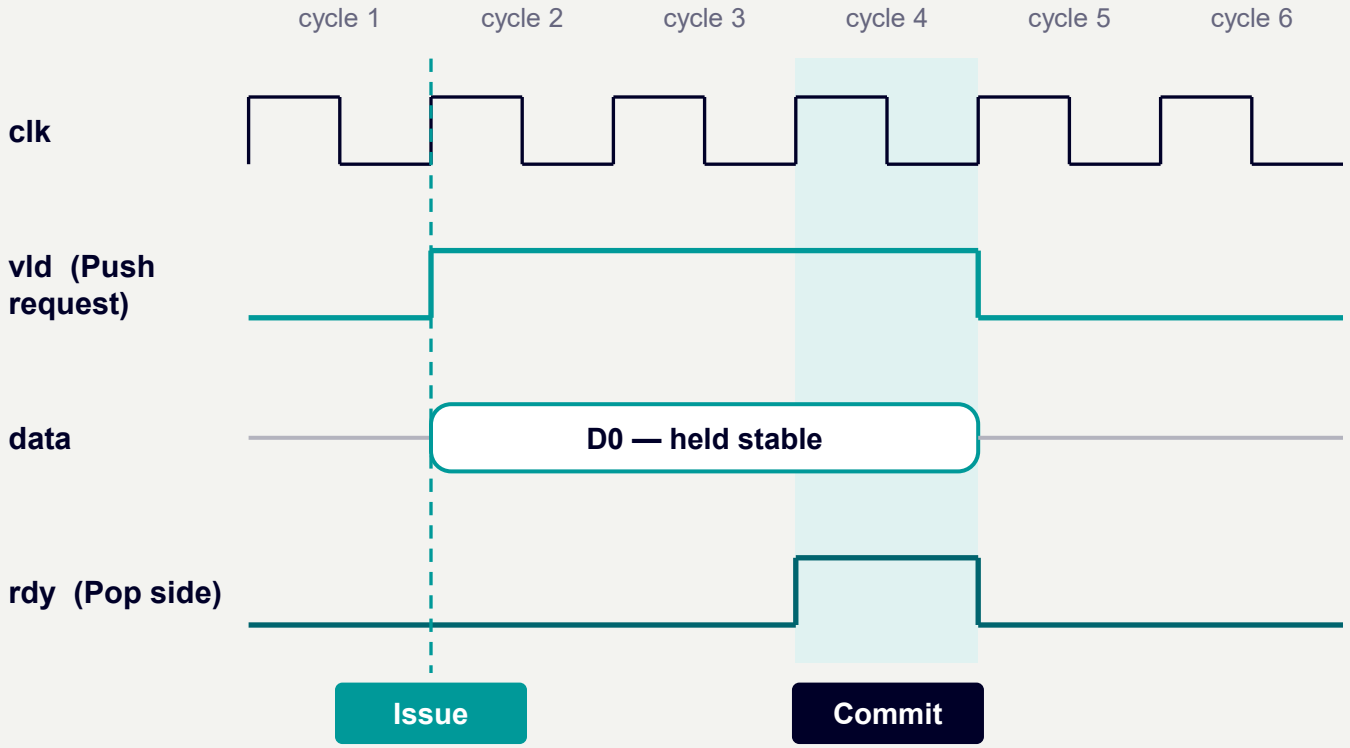


Message calls never move a signal's anchor, and signal statements add no message-order constraints. No "protocol regions" are needed.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Issue and commit: the two moments of a transfer



Blocking Push: valid and payload are held from issue until both valid and ready are high at the commit edge.

Issue

The cycle the process starts requesting: valid for a Push, ready for a Pop.

Commit

The cycle in which valid and ready are both high; the payload is the data seen then.

Persistent requests

A blocking request stays asserted, payload stable, until commit or reset — no try-and-withdraw.

Registered results

A result returned at a cycle boundary cannot drive a dependent request in that same cycle.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

The basic conceptual model in four rules

1

Synchronization calls stay in source order.

wait(), SyncChannel and ac_sync are never reordered within a process.

2

Signal reads occur at the closest preceding synchronization call.

A read observes the value present at that anchor.

3

Signal writes occur at the closest succeeding synchronization call.

That is when the value becomes visible to others.

4

Message passing may shift, share a cycle or overlap…

…within the ordering and boundary constraints on the next slide.

Scope

Processes without loop pipelining. Preserved (rolled) loops are allowed and may contain explicit waits.

Key idea

Everything HLS adds — extra states, pipeline stages, late sampling — must remain explainable by these four rules at the process interfaces.

Public | © Siemens 2026 | Siemens Digital Industries Software

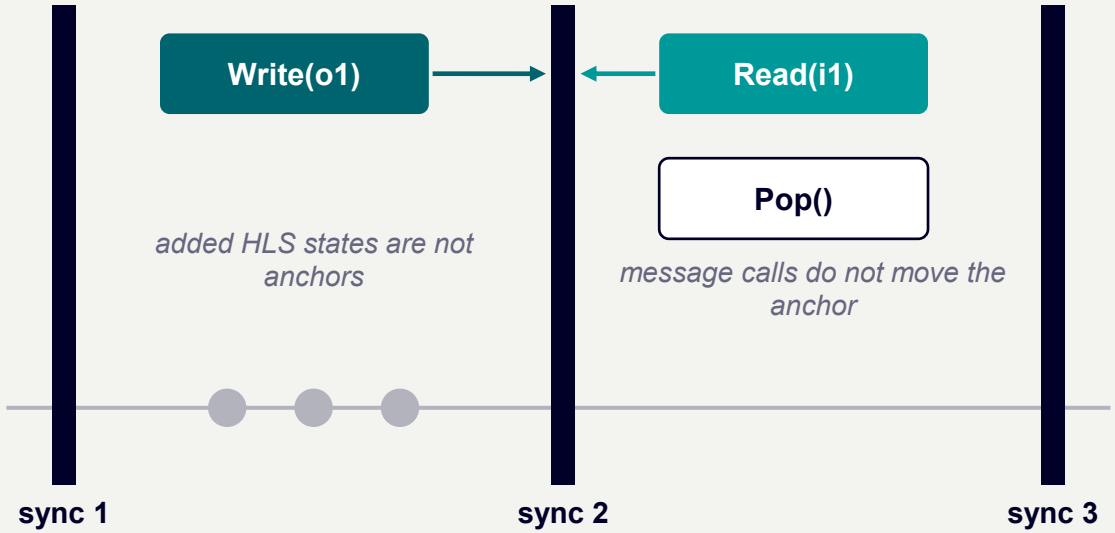
SIEMENS

Signal IO is anchored to synchronization points

```
while (1) {
  o1.write(new_val);        // Write -> next wait
  do {
    wait();
  } while (!go.read());
  new_val = i1.read();      // Read -> prior wait
  new_val = f(new_val);     // no internal IO
}
```

- Read(sig) is anchored to the closest preceding sync call; Write(sig) to the closest succeeding one.
- Several writes before one anchor: the last one supplies the value.
- Same convention as Verilog: inputs sampled at the edge, outputs visible at the next edge.

One process, logical timeline



The physical sample may move; the logical Read or Write does not.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Message passing: overlap is fine; reversal and boundary-crossing are not

Before a sync

Issued no later than the cycle the sync commits; blocking calls complete by then.

After a sync

Issued only after the cycle in which the sync commits.

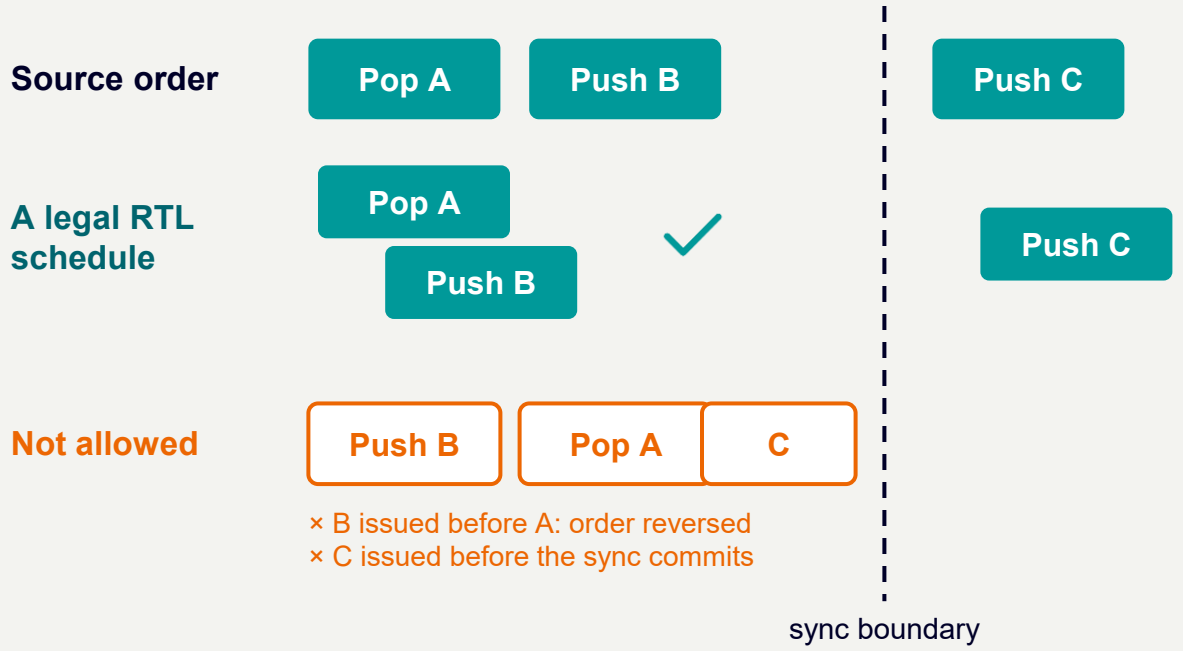
Same interface

Issued in source order — also across overlapped pipeline iterations.

Different interfaces

May overlap or share a cycle, but never issued in reverse source order.

Source order versus RTL issue



The sync is a fence: no earlier blocking request survives into the next interval, though a message and its sync may complete on the same edge.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

What the no-reverse rule buys — and what it does not

Guaranteed by the default contract

HLS cannot introduce a new deadlock merely by reversing source-ordered message calls.

Same-interface order and distinct-interface no-reverse order both hold in the RTL, including across overlapped pipeline iterations.

Reversal — the factory deadlock — is ruled out by construction.

Not covered by this rule alone

Loop overlap, eager issue, finite capacities and the environment can still affect deadlock.

The separate completion result (Appendix K) handles these, under its capacity, correspondence and progress premises.

Its conclusion is acceptance of the required workload — not freedom from every internal wait cycle.

`STRICT_IO_SCHEDULING=relaxed`

Lets HLS reorder calls on distinct interfaces (never across sync). May add deadlocks; the formal results assume it is off.

1. **Run relaxed to see the order HLS prefers for QOR**
2. **Rewrite the source in that order**
3. **Return to the default scheduling mode**

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

What HLS may do — and why the interfaces stay compatible

Transformation	What HLS may do	Why interface behavior is preserved
Added states	Insert cycles anywhere in a process, e.g., to share resources	Extra states are silent internal steps; the order of observable events is unchanged
Loop pipelining	Overlap loop iterations to raise throughput	Stages are internal; channel order, issue order and sync boundaries still hold
Direct inputs	Sample stable signals late, without capture registers	The value is stable, so a late sample equals the value at the anchor
Memory reordering	Move array accesses when proven conflict-free	Logical memory behavior and sync fences are preserved at the declared boundary
Finite buffering	Implement channels with bounded storage	The reference model is configured with the same back-annotated capacities
Cycle-exact modes	`IMPLICIT_FSM=true` or `IO_MODE=FIXED`	Interface cycles identical to pre-HLS; the internal FSM is identical only with IMPLICIT_FSM



Default: HLS may add latency. Cycle-exact modes trade that freedom for identical interface timing.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Pipelined loops

Overlapped iterations, initiation interval 1



Orange lines: synchronization calls stay outside the pipelined loop body — before and after it.

No sync inside the loop body

No wait, ac_wait, SyncChannel or ac_sync in a pipelined loop. Put them outside, or don't pipeline that loop.

Same interface, same order

Accesses to one channel, signal or sync interface keep their order across iterations.

Signal IO stays anchored

Signal accesses belong to the surrounding sync calls, not to pipeline stages.

Changes are flagged

If pipelining would reorder signal IO or split same-edge IO, the tool requires an explicit pragma (Cat7).

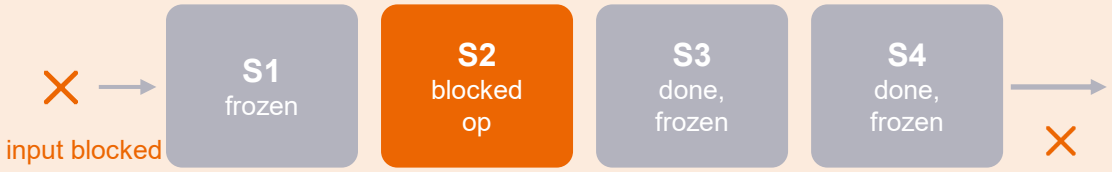
Bottom line: if a process uses only latency-insensitive protocols, pipelining by default breaks none of them.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

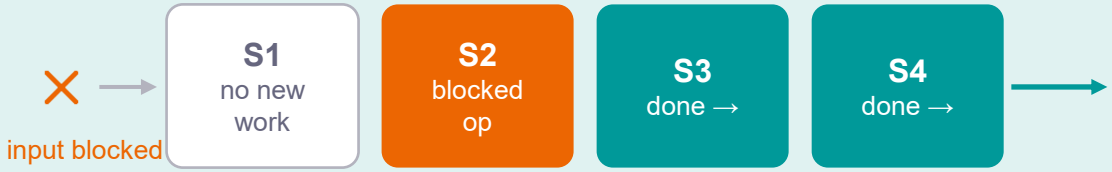
Pipelines must flush, not freeze

Global stall: does not qualify for completion



Everything stops: completed downstream work cannot leave while upstream is blocked.

Ordered automatic flush (B.OF)



No younger iteration launches past the blocked operation; completed downstream work keeps draining.

Block point

Blocks at the same pending operation as the unpipelined source.

No new later work

No younger iteration launches or issues past the blocked frontier.

No withdrawal

Asserted blocking requests stay asserted until they commit.

Ordered drain

Completed downstream work drains, in order, regardless of upstream.

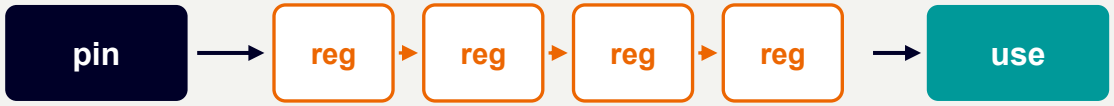
`hls_stall_mode flush` is a request, not evidence: the generated controller must actually meet B.OF / B.CT.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

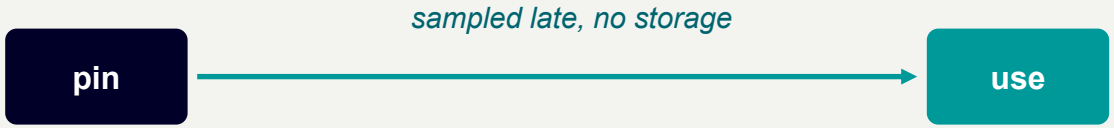
Direct inputs: stable signals, sampled late

Ordinary signal read



Read at its anchor, then carried through added states or stages: one register per read, per stage.

```
#pragma hls_direct_input
```



Logically, the Read still happens at its anchor. Physically, HLS may sample the stable signal whenever it needs the value.

The contract

The environment holds the signal stable once the receiving processes leave reset, until the next reset affecting them.

The freedom

HLS samples physically as late as it likes, with no capture registers — saving area.

The observation

Late samples are internal: no extra Read events, and the attributed value cannot change.

Dynamic resets

Allowed. Each reset starts a new stability epoch for the affected logic and its direct inputs.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Changing configuration safely: hls_direct_input_sync

```
#pragma hls_direct_input
sc_in<uint32_t> direct_inputs[N];
...
while (1) {
  #pragma hls_direct_input_sync all
  sync_in.sync_in();     // update point
  #pragma hls_pipeline_init_interval 1
  #pragma hls_stall_mode flush
  for (x = 0; x < direct_inputs[0]; x++)
    for (y = 0; y < direct_inputs[1]; y++) {
      sum = 0;
      for (s = 0; s < num_samples; s++)
        sum += sample_in[s].Pop()
               * direct_inputs[2 + s];
      ...  out1.Push(result);
    }
}
```

Abridged from Catapult Matchlib example 61

Only sync may gate these updates — never Push/Pop or ac_channel.

1. **DUT asserts sync rdy**

   Only once old values are no longer needed — pipeline ramped down, or remaining uses protected.

2. **TB changes inputs, asserts vld**

   After seeing rdy; the new values settle before the sampling edge.

3. **Sync completes at the edge**

   rdy and vld both high. One cycle of vld suffices — never drop it just because a cycle passed.

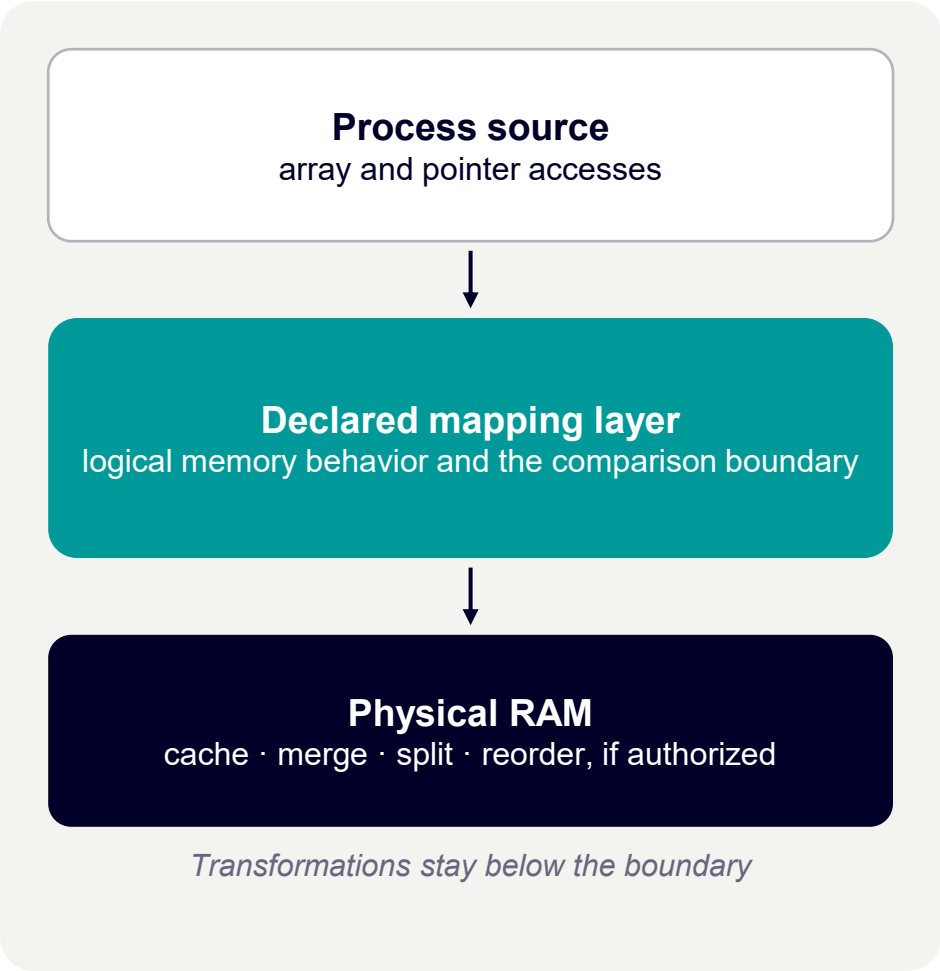
4. **Inputs stay stable**

   Until the next permitted update or an affecting reset.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Arrays and memories



Declare the mapping

Name the logical behavior, the comparison boundary and how it is represented: signals, messages, sync or a qualified memory interface.

Transform only below it

No visible interaction created, dropped, duplicated, relabeled or illegally reordered.

Respect sync fences

Accesses before a sync commit no later than the sync; never merge accesses across one.

Expose cross-process values

If one process's write affects another's control, data or order, represent that dependency (J.Mem).

Count backpressure

A memory that can stall a process belongs in the completion qualification.

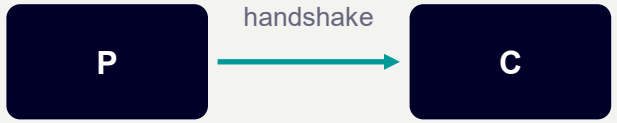
Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Channel capacity belongs in the reference model

Rendezvous

`B(c) = 0`



Push and Pop commit together: same cycle, same value.

Bounded FIFO

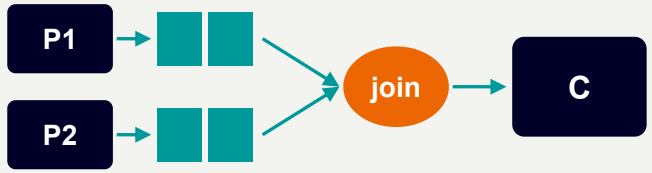
`B(c) > 0`



Pop needs an item at the start of the cycle; Push needs space then. No fall-through within a cycle.

Elaborated structure

joins, staging, gates



Explicit finite storage plus stateless ready/valid couplers, when one depth is not faithful.

The reference model is not the raw simulation

Raw SystemC
unadjusted channel depths

≠

Reference model Sys_ref
source + testbench with back-annotated capacities, arbitration and coupling

Matchlib note

In<> ports model a one-place skid buffer by default — right for throughput modeling.

For final verification, match ports and capacities to the selected RTL and re-establish faithful capacities, e.g. with

```
-DFORCE_AUTO_PORT=Connections::SYN_PORT
```


Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

RULE 1: every signal access needs one unambiguous anchor

Risky **A blocking Pop splits read from wait**

```
do { wait(); } while (!go.read());
int pop_val = pop1.Pop(); // may stall
new_val = i1.read();  // i1 may have
                      // changed by now
```

Fix: read i1 immediately after its wait. No cost in QOR or simulation speed.

Ill-formed **One write, two possible anchors**

```
wait();                       // WAIT 1
o1.write(some_value);
if (go.read()) { ... }
else { wait(); }              // WAIT 2
some_other_function();
wait();                       // WAIT 3
```

o1 binds to WAIT 2 or WAIT 3 depending on go. Flag as an error; move the write next to its intended wait.

RULE 1 Group signal writes just before their wait and reads just after it. An unresolved warning is not enough to claim the guarantees: fix, justify or exclude.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

RULE 2: signal IO inside rolled loops

```
wait();              // start of while loop
#pragma hls_unroll no
for (int i = 0; i < 10; i++) {
  o1.write(i1.read() * i); // no anchor
}                          // in the loop
```

Why it is a problem

- With no sync inside the loop, every iteration shares one pair of anchors.
- Logically there is one Read; only the final Write supplies the value at the next anchor.
- An RTL that samples a fresh i1 per iteration differs from the anchored meaning. Added states never create anchors.

Per-iteration IO, loop not pipelined

Place a wait() in the loop body — and follow RULE 1.

One value intended

Move the read before the loop to make the intent explicit.

Loop will be pipelined

No in-body wait allowed: move IO outside, use message passing, use hls_direct_input(_sync), or don't pipeline.

RULE 2 Signal IO in a rolled loop needs a wait in the loop body.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Three common sources of pre-HLS / post-HLS mismatch

Non-blocking calls

PushNB / PopNB / nb_read / nb_write outcomes depend on latency — which HLS changes.

Instead

Use only when needed, e.g., time-based arbitration at a shared resource.

One-way messages as sync

Latency and storage between processes vary, so one channel does not synchronize two processes.

Instead

Use SyncChannel, or two opposite channels (AXI ar/r, aw/b) — weaker, but valid.

Signals without a handshake

wait()-synchronized signal IO without a two-way handshake is likely latency-sensitive.

Instead: a two-way handshake

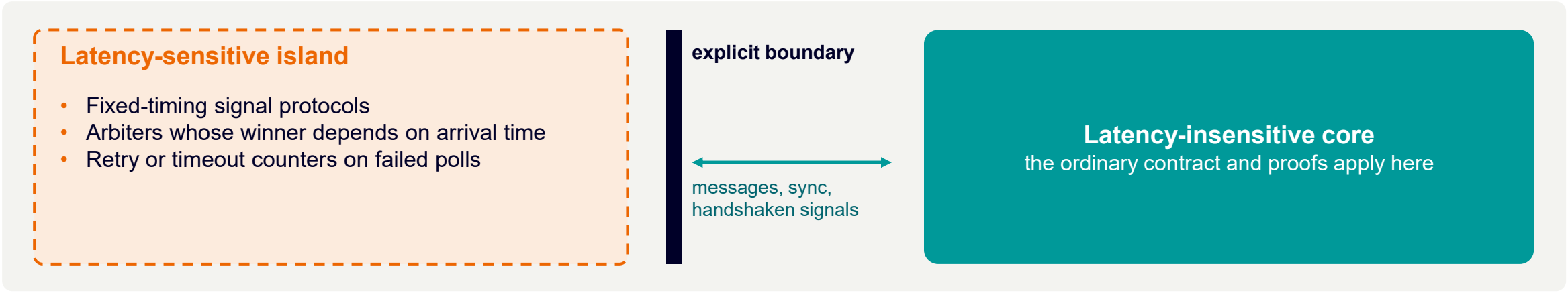
```
out_dat = value;
out_vld = 1;
do {
  wait();
} while (out_rdy != 1);
out_vld = 0;
```

Rule of thumb: if behavior depends on how long something took, it is latency-sensitive — synchronize explicitly or isolate it.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Isolating latency-sensitive behavior



Transactors

App. D

One side handles detailed signal timing (cycle-accurate SystemC); the other exchanges messages. See example 53_transactor_modeling.

One-way handshakes

App. E

Use only where back-pressure cannot occur, e.g., fixed-rate dat/vld streams (example 32), and assert that it never does.

Snooping

App. L

Run RTL alongside the source; release pre-HLS arbiter inputs or interrupts only when the RTL shows them, so both pick the same winners.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Modeling guidelines at a glance

In general

- Prefer `Connections::In/Out` + `SyncChannel` over signal IO and wait()
- Prefer `Pop()/Push()` over `PopNB()/PushNB()`
- For pipelined loops in completion claims, check that the generated controller meets the flush laws
- Unless modeling cycle-accurately, expect latency to differ between pre- and post-HLS models

When using signal IO

- Cycle-accurate process: use `disable_spawn` and the example-53 transactor style, with no Push/Pop
- Direct inputs: `hls_direct_input` (+ `_sync`); change only at allowed times
- Mixing signals with In/Out: use a real signal handshake, not the ports, for synchronization
- Keep signal IO right next to its wait()

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Two guarantees with different jobs

Safety

implementation → reference

Covered RTL behavior introduces no observable functional behavior absent from the prescribed reference model.

Needs

- Scheduling rules R1–R5 / E1–E5
- Local process and channel proofs
- Faithful capacities, environment and reset conditions
- An independent proof of the property on the source

`J.LG-Origin · J.BatchPrefixCoverage · J.SafeTransfer-HistCov`

Completion

required workload

The required outputs eventually arrive with the expected values and per-interface order.

Needs

- A finite accepting source run (serial witness)
- Acyclic dependency closure
- Internal-fact correspondence and value agreement
- Pipeline / resource progress and service assumptions

`K.CT.Completion · K.CT.Standard · B.OF · B.CT`

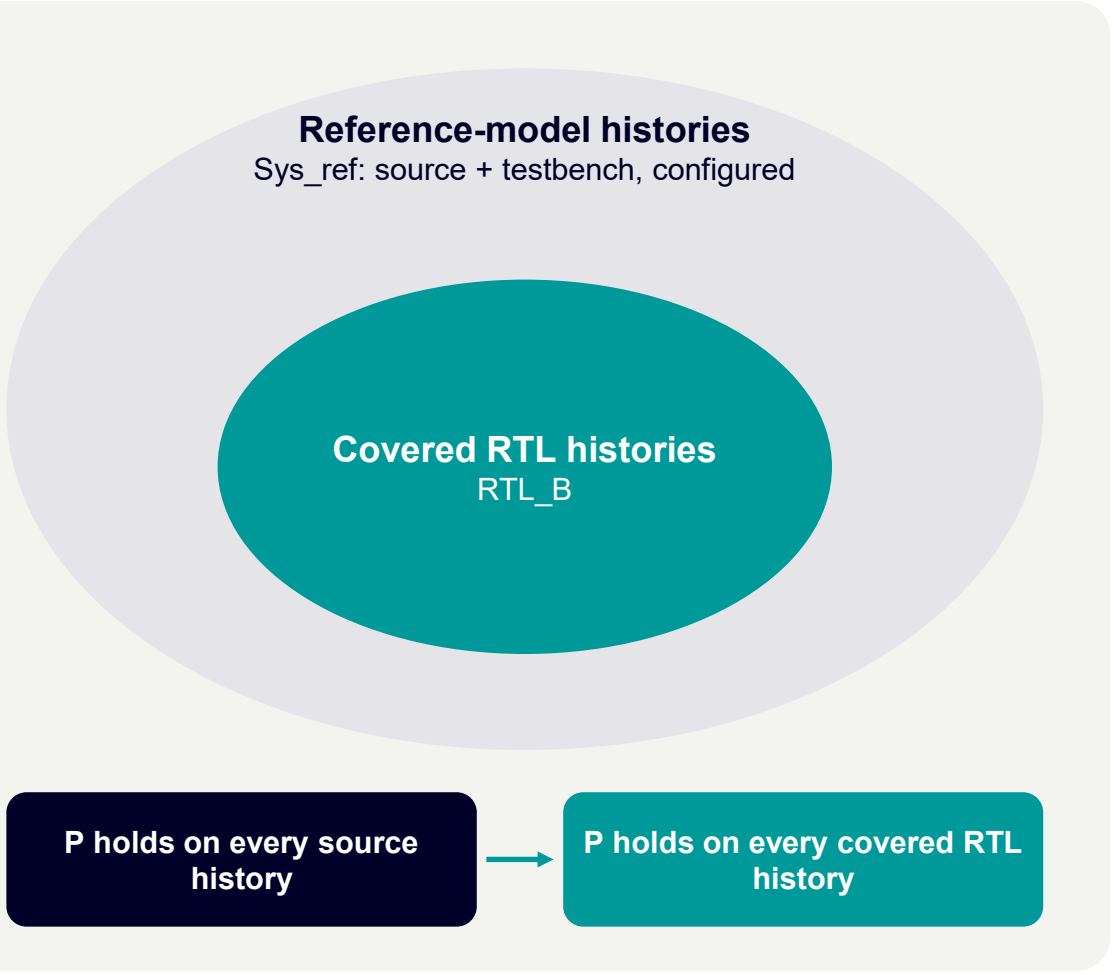
Neither implies the other: safety promises no output; completion promises no cycle bound.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Rules: App. G · J.ReachableHistCov · J.SafeTransfer-HistCov

Safety: every covered RTL behavior has a source explanation



Direction matters

RTL → source. A particular pre-HLS run is not guaranteed to be reproduced by the RTL.

“Equal” means equal history

Same events, values and required order at the selected interfaces — not the same cycles, registers or grouping.

Write properties on that history

A scoreboard matching requests and responses by value and order transfers; a raw cycle-count check does not.

The source property is separate

It must hold for all admitted source behaviors. One passing simulation is not that proof.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Rules: Appendix G

The trace rules: R1–R5 for the source, E1–E5 for the RTL

Topic	Source rule (pre-HLS reference)	Implementation rule (post-HLS RTL)
Sync order	R1 · Sync calls occur in program order	E1 · Per-process sync order preserved
Signal anchoring	R2 · Reads at the preceding sync, writes at the succeeding sync. R2C: combinational logic observed at its settled fixed point	E2 · Signal visibility preserved at the same anchors
Isolation around sync	R3 · Messages stay in their interval; blocking ones complete by the sync	E5 · Messages cannot cross their immediate sync boundary
Message issue order	R4 · Never issued in reverse source order	E3 · Same ordering, prefix-closed, also across overlapped iterations
Channel semantics	R5 · Rendezvous (B = 0) or bounded FIFO (B > 0)	E4 · No drops, duplicates or reordering; occupancy stays within 0…B



τ_ref ≈ τ_post is an ordered compatibility relation — source witness on the left, RTL on the right — not a symmetric equivalence.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

How local proofs compose into a system result

1 Local evidence

Per process, channel and atomic interaction: source and RTL steps correspond.

`contracts C0–C5`

2 Integration checks

The ready/valid network settles; environment, reset and capacities are faithful.

`settling · environment`

3 Generic composition

Induction over actual RTL steps builds a legal source prefix for every RTL prefix.

`J.LG-Origin · J.BatchPrefixCoverage`

4 Safety transfer

A property proved separately on the source carries over to the RTL.

`J.SafeTransfer-HistCov`

Why it matters

Local evidence replaces an intractable whole-system proof. For ordinary profiles, no design-specific global invariant is needed (J.OrdinaryConstructiveComposition).

What it does not assume

No fairness and no future cycles: deadlocked and terminated prefixes are covered too. It gives safety only — no progress conclusion.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Testbenches, properties and stimulus classes

The testbench is in the system

Reactive responses that depend on earlier DUT results are covered by the same joint argument.

One proof for a class of stimuli

A reusable correspondence proof can cover a declared class of inputs and choices; each new stimulus needs only binding and membership checks.

The source property is your job

It must hold for all admitted source behaviors. A passing sample campaign does not establish that.

Latency-sensitive testbenches

Branches, watchdogs or pass/fail decisions on raw cycle counts are not covered automatically: isolate, make explicit, or check at RTL.

Theorems: J.UniformConstructiveCoverage, J.ConstructiveClassSafe. Reuse can cut effort across a large, stable campaign — with no unconditional cost advantage.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Completion: proving the required outputs arrive

1

Declare the workload

The testbench acceptances you require, with values and per-endpoint order.

2

Run a serial witness

A finite, conservative source run that accepts them, with real capacities.

3

Close the graph

Acyclic prerequisites: required work, control decisions (even untaken branches), space.

4

Bind to RTL

Internal-fact correspondence and value agreement; source-aligned control.

5

Show progress

Pipelines flush (B.OF), resources persist (B.CT), service and fairness hold (App. N).

`K.CT.Completion` Every covered RTL execution that satisfies the service assumptions eventually accepts the workload with the same values and per-endpoint order.

Serial = conservative

Within a sync interval, a later blocking call waits until every earlier one has committed.

Failure is inconclusive

A failed witness or cyclic graph is not a deadlock verdict.

Not claimed

No cycle bound; unrelated or post-completion deadlocks are outside the result.

Public | © Siemens 2026 | Siemens Digital Industries Software

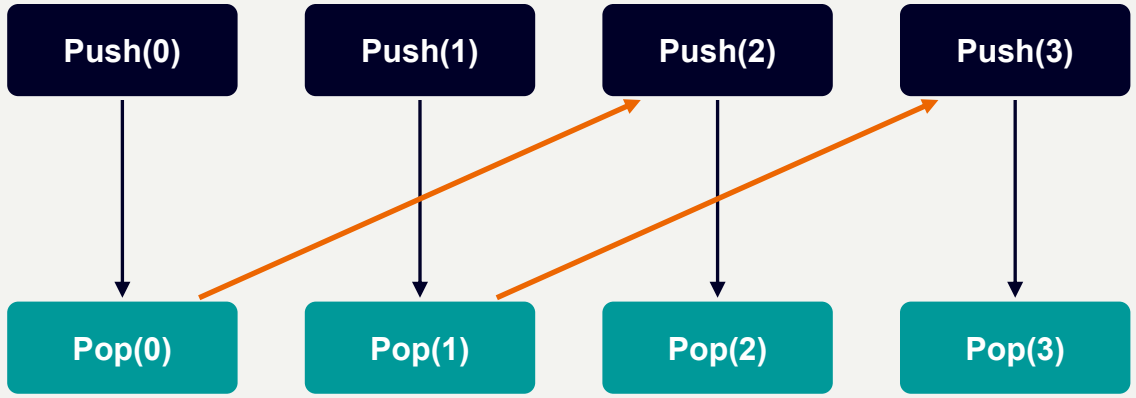
SIEMENS

Guaranteed capacity, not nominal capacity

Nominal capacity 4, guaranteed floor g = 2



2 guaranteed slots
2 nominal, not guaranteed (reserved / pipeline storage)



Orange space edges: Pop(k − g) must precede Push(k) for k ≥ g — even if the source run used all four slots.

Use the floor

Witness space dependencies use the guaranteed floor g, never the nominal capacity.

Reserved space does not count

Reserved slots and storage inside a pipeline do not enlarge g automatically.

Watch k = g

The boundary case is the edge most easily lost.

Unbounded is not evidence

A run with unbounded channels says nothing about finite hardware.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

What the guarantees do not claim

Reverse realization

The RTL need not reproduce an arbitrarily chosen source simulation.

Cycle equality

No identical timing, registers, latency or throughput — those need their own timing model and evidence.

A cycle bound

Completion means “eventually”, not “within N cycles”.

Total deadlock freedom

Deadlocks in unrelated work, or after the required outputs are accepted, are outside the result.

Automatic tool certification

The proofs do not certify a particular HLS run; the flow must supply the evidence.

Multiple clocks, async reset

One global clock per theorem instance; asynchronous reset assertion needs a separate argument.

When a sufficient check fails, the result is inconclusive — it does not prove the design wrong.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Design constraints for architects

1 Anchor every signal access

One unambiguous sync anchor for every read and write.

2 Stable direct inputs

Stable per reset epoch, or updated only via the sync handshake.

3 Explicit timing sensitivity

Latency-dependent environment behavior isolated or made observable.

4 Polling is a design decision

Non-blocking control picks a proof route; prefer blocking calls.

5 No unbounded busy polling

Qualify retries under the chosen profile, or use a blocking call.

6 Expose shared-memory values

Each cross-process memory dependency via a supported boundary.

7 Model the real channel

No hidden fall-through, bypass, debug drain or side entrance.

8 Elaborate joined structures

Joins, coupled multi-input pipelines and sync gates modeled explicitly.

9 Reset with a disposition

Reset both channel ends together, or define a flush / abort protocol.

For completion, also: point-to-point channels with explicit arbiters · explicit multi-lane transfers · persistent requests · real capacities · explicit end-of-stream and abort protocols

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

Who supplies which evidence

Framework — once

Generic construction, composition and workload-induction theorems

Tool or component family — once per family

Qualified transformations, channel primitives, controller laws, runtime semantics

Design or stimulus class — per design

Local process invariants, integration and environment evidence, source-property proofs

Concrete application — per instance

Bind generated artifacts, topology, parameters, boundaries and workloads

The division of labor

- Architects choose interfaces, reset and communication structures that admit the correspondence.
- The HLS tool and verification flow build and check the detailed proof records.
- Evidence: documentation, lint and static checks, targeted RTL / interface checks, formal verification.
- Written with the precision intended to support mechanization in Lean.

Most of these conditions were always there. The formalization names them, so they can be audited instead of quietly assumed.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

No surprises, by construction

Simple rules

Three IO classes, one uniform rule each

Freedom for QOR

Pipelining, added states, late sampling, buffering

Verify at the source

One testbench; interface properties that transfer

Backed by proofs

Local evidence composes into system guarantees

Most functional verification and debug happen at the source · HLS keeps the freedom to optimize
A tool-neutral contract · a starting point for standardization in the Accellera Synthesis Working Group

Write latency-insensitive, well-anchored models — and the RTL holds no surprises at the interfaces.

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS

References and further reading

Repository github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG

Catapult HLS User View Scheduling Rules (23 October 2026)
…/matchlib_examples/doc/catapult_user_view_scheduling_rules.pdf

Formal Architecture Guide (23 October 2026)
…/matchlib_examples/doc/formal_architecture_guide.pdf

Brief summary of the scheduling rules
…/matchlib_examples/doc/brief_scheduling_rules.txt

Examples cited: 09, 12, 15, 32, 53, 61
…/matchlib_examples/examples

Making shift-left real (white paper)
hls.academy/making-shift-left-real-why-systemc-matchlib-and-high-level-synthesis-…

Verifying hardware at the source (white paper)
hls.academy/topics/systemc-modeling/verifying-hardware-at-the-source-…

Reading paths in the rules

Architects and HLS users

Main body through App. F; App. B and C; App. K for completion

Safety reviewers

Normative Formal Core; App. G; App. J constructive theorems

Overview and evidence

App. M formal results; M.7a evidence checklist

Public | © Siemens 2026 | Siemens Digital Industries Software

SIEMENS